

Student Grade Search & Sort System

Algorithm Design & Analysis — Project Report

Language: Java | Algorithms: Bubble Sort · Linear Search · Binary Search

Presented by : Hadi Hussein Muselmani (202304319)

Course Instructor : Dr.Julie Abu Daher

Course TA: Miss Fatima El Harakeh

1. PROJECT DESCRIPTION

This project implements a simple Student Grade Management System in Java. The program allows a user to enter a list of student names and their scores. It then demonstrates two classic algorithms — sorting and searching — on that data, and finishes with a performance comparison showing why algorithm choice matters.

The program does the following steps in order:

- Collects student names and scores from the user (keyboard input).
- Displays the original, unsorted list.
- Runs Linear Search on the unsorted list to find a target score.
- Sorts the list using Bubble Sort (low score to high score).
- Runs Binary Search on the now-sorted list to find the same target score.
- Prints a comparison table showing Big-O complexity and actual execution time.

The project is split across three files: Student.java (data class), SortSearch.java (all algorithm methods), and Main.java (program entry point and output).

2. ALGORITHMS USED

Algorithm	Type	Time Complexity	Requirement
Bubble Sort	Sorting	$O(n^2)$	None — works on any list
Linear Search	Searching	$O(n)$	None — works on any list
Binary Search	Searching	$O(\log n)$	List must be sorted first

3. HOW THE SOLUTION WORKS

Bubble Sort

Bubble Sort works by going through the list of students repeatedly. During each pass, it compares two neighbouring students. If the one on the left has a higher score than the one on the right, they are swapped. After each full pass, the student with the highest remaining score has 'bubbled up' to its correct place at the end. The process repeats until no more swaps happen, meaning the list is fully sorted. Because of the two nested loops, its time complexity is $O(n^2)$.

Linear Search

Linear Search starts at the very first student in the list and checks each one in order until it either finds the target score or reaches the end. It does not require the list to be sorted. In the worst case it checks every student, giving a time complexity of $O(n)$.

Binary Search

Binary Search requires the list to be sorted first. It looks at the middle student: if that student's score equals the target, the search is done. If the target is smaller, it searches only the left half; if larger, only the right half. This halving continues until the target is found or the search area is empty. Because it eliminates half the remaining students on every step, its time complexity is $O(\log n)$ — much faster than Linear Search for large lists.

4. PSEUDOCODE

Bubble Sort

```
BUBBLE SORT (students[ ], n)
  FOR i FROM 0 TO n-2:
    swapped = FALSE
    FOR j FROM 0 TO n-2-i:
      IF students[j].score > students[j+1].score THEN
        SWAP students[j] WITH students[j+1]
        swapped = TRUE
      END IF
    END FOR
    IF swapped = FALSE THEN STOP    // list is already in order
  END FOR
END
```

Linear Search

```
LINEAR SEARCH (students[ ], n, target)
  FOR i FROM 0 TO n-1:
    IF students[i].score = target THEN
      RETURN i                // position where target was found
    END IF
  END FOR
  RETURN -1                  // target not found
END
```

Binary Search (list must be sorted first)

```
BINARY SEARCH (students[ ], n, target)
  low  = 0
  high = n - 1
  WHILE low <= high:
    mid = (low + high) / 2
    IF students[mid].score = target THEN
      RETURN mid                // found at the middle position
    ELSE IF students[mid].score < target THEN
      low = mid + 1             // target is in the right half
    
```

```

ELSE
    high = mid - 1        // target is in the left half
END IF
END WHILE
RETURN -1                // target not found
END

```

5. PERFORMANCE DISCUSSION

For a small class of students the difference in execution time between Linear Search and Binary Search is barely noticeable. However, as the number of students grows, the gap becomes significant. Linear Search grows linearly — doubling the list doubles the work. Binary Search grows logarithmically — doubling the list adds only one extra step. For example, searching 1,000 students takes up to 1,000 comparisons with Linear Search but at most 10 comparisons with Binary Search.

The trade-off is that Binary Search requires the list to be sorted first. Bubble Sort has a cost of $O(n^2)$, which is high for very large lists. In practice, this means Binary Search is the better choice when the same sorted list will be searched many times, while Linear Search is simpler and sufficient for small or unsorted data.

Output of code

```

Main [Java Application] C:\Users\HP\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jre\bin\java.exe
=====
STUDENT GRADE SEARCH & SORT SYSTEM
Algorithms: Bubble Sort, Linear &
           Binary Search
=====

How many students do you want to enter? 5

Enter each student's name and score:
Student 1 name : Hadi
Student 1 score: 90

Student 2 name : Ali
Student 2 score: 87

Student 3 name : Hasan
Student 3 score: 76

Student 4 name : Hussein
Student 4 score: 56

Student 5 name : mohd
Student 5 score: 34

```

Step 1: input

```
--- ORIGINAL LIST (unsorted) ---  
-----  
No.   Name           Score  
-----  
1     Hadi           90  
2     Ali            87  
3     Hasan          76  
4     Hussein        56  
5     mohd           34  
-----
```

Step 2 : table format (unsorted)

```
Enter the score you want to search for: 90  
  
--- LINEAR SEARCH (on unsorted list) ---  
Found! Hadi has a score of 90 (position 1 in unsorted list)  
Time taken: 1234900 nanoseconds  
  
--- BUBBLE SORT (sorting by score, low to high) ---  
Sorted list:  
-----  
No.   Name           Score  
-----  
1     mohd           34  
2     Hussein        56  
3     Hasan          76  
4     Ali            87  
5     Hadi           90  
-----  
--- BINARY SEARCH (on sorted list) ---  
Found! Hadi has a score of 90 (position 5 in sorted list)  
Time taken: 10900 nanoseconds  
  
=====
```

PERFORMANCE COMPARISON		
Algorithm	Complexity	Time (ns)
Bubble Sort	$O(n^2)$	N/A (sort step)
Linear Search	$O(n)$	1234900
Binary Search	$O(\log n)$	10900

```
-----  
  
DISCUSSION:  
    Linear Search checks students one by one – no sorting needed.
```

Step 3 : searching